

SP-4 Phase F — Clone searchable (implementation plan)

Plan date: 2026-05-13 **Design:** docs/superpowers/specs/2026-05-13-sp4-phase-f-design.md **Prerequisite:** HEAD 657f74c1 (Phase D landed).

5 tasks, ~15 steps. Single-session executable.

Task 1 — ClonedRoutingTrackerClient

Files:

- Create: core/tracker/client/src/main/kotlin/lava/tracker/client/ClonedRoutingTrackerClient.kt

Steps:



Step 1: Class signature

```
internal class ClonedRoutingTrackerClient(
    private val sourceClient: TrackerClient,
    private val cloneDescriptor: ClonedTrackerDescriptor,
) : TrackerClient {
    override val descriptor: TrackerDescriptor = cloneDescriptor
    override suspend fun healthCheck(): Boolean =
        sourceClient.healthCheck()
    override fun close() = sourceClient.close()

    override fun <T : TrackerFeature> getFeature(featureClass:
        KClass<T>): T? {
        val sourceFeature =
            sourceClient.getFeature(featureClass) ?: return null
        @Suppress("UNCHECKED_CAST")
        return when (featureClass) {
            SearchableTracker::class ->
                wrapSearchable(sourceFeature as SearchableTracker) as T
            BrowsableTracker::class ->
                wrapBrowsable(sourceFeature as BrowsableTracker) as T
            FavoritesTracker::class ->
                wrapFavorites(sourceFeature as FavoritesTracker) as T
        }
    }
}
```

```

        // Topic / Comments / Authenticatable / Downloadable
        carry no
        // user-visible "tracker id" field – return source's
        impl unchanged.
        else -> sourceFeature
    }
}

private fun wrapSearchable(src: SearchableTracker):
    SearchableTracker = object : SearchableTracker {
        override suspend fun search(request: SearchRequest, page:
            Int): SearchResult {
            val raw = src.search(request, page)
            return raw.copy(items = raw.items.map {
                it.copy(trackerId = cloneDescriptor.trackerId) })
        }
    }

private fun wrapBrowsable(src: BrowsableTracker):
    BrowsableTracker = object : BrowsableTracker {
        override suspend fun browse(category: String?, page:
            Int): BrowseResult {
            val raw = src.browse(category, page)
            return raw.copy(items = raw.items.map {
                it.copy(trackerId = cloneDescriptor.trackerId) })
        }
        override suspend fun getForumTree(): ForumTree? =
            src.getForumTree()
    }

private fun wrapFavorites(src: FavoritesTracker):
    FavoritesTracker = object : FavoritesTracker {
        override suspend fun list(): List<TorrentItem> =
            src.list().map { it.copy(trackerId =
                cloneDescriptor.trackerId) }
        override suspend fun add(id: String): Boolean =
            src.add(id)
        override suspend fun remove(id: String): Boolean =
            src.remove(id)
    }
}

```



Step 2: Compile.

Task 2 — `LavaTrackerSdk.clientFor(id)` + replace all `registry.get` call sites

Files:

- Modify: `core/tracker/client/src/main/kotlin/lava/tracker/client/LavaTrackerSdk.kt`

Steps:



Step 1: Add `clientFor` helper.

```
private suspend fun clientFor(id: String): TrackerClient {
    val dao = clonedProviderDao
    if (dao != null) {
        val cloned = dao.getAll().firstOrNull { it.syntheticId == id }
        if (cloned != null) {
            val sourceDescriptor = registry.list().firstOrNull {
                it.trackerId == cloned.sourceTrackerId }
            ?: throw IllegalStateException(
                "Clone $id references unknown source $
                {cloned.sourceTrackerId}",
            )
            val cloneDescriptor = ClonedTrackerDescriptor(source
                = sourceDescriptor, override = cloned)
            val sourceClient =
                registry.get(cloned.sourceTrackerId, MapPluginConfig())
            return ClonedRoutingTrackerClient(sourceClient,
                cloneDescriptor)
        }
    }
    return registry.get(id, MapPluginConfig())
}
```



Step 2: Replace `registry.get(id, MapPluginConfig())` call sites.

Use `grep` to locate; each call site that's inside a suspend method gets replaced with `clientFor(id)`. The non-suspend `getActiveClient()` is preserved as a compatibility shim that does `runBlocking { clientForActive() }` (mirrors the existing `listAvailableTrackers runBlocking` pattern); a new suspend `fun getActiveClientSuspend()` is added for callers that already suspend, and the suspending public methods (`search`, `browse`, `multiSearch`, etc.) use the suspend overload.



Step 3: Compile ./

```
gradlew :core:tracker:client:compileDebugKotlin  
spotlessKotlinCheck.
```

Task 3 — Toast copy disclosure

Files:

- Modify: feature/provider_config/src/main/kotlin/lava/provider/config/ProviderConfigViewModel.kt

Steps:



Change ProviderConfigSideEffect.ShowToast("Cloned") to ProviderConfigSideEffect.ShowToast("Cloned (URL routing pending – searches use source URLs)").



Compile.

Task 4 — Unit tests

Files:

- Create: core/tracker/client/src/test/kotlin/lava/tracker/client/LavaTrackerSdkCloneSearchTest.kt

Steps:



Step 1: Test fake ClonedProviderDao. In-memory list-backed implementation with getAll() / upsert / delete / observeAll.



Step 2: Test 1 — clone search does NOT crash.

- Build a DefaultTrackerRegistry with a fake "rutracker" factory.
- Build a fake ClonedProviderDao containing one clone row (syntheticId="rutracker.clone.x", sourceTrackerId="rutracker", displayName="RuTracker EU", primaryUrl="https://rutracker.eu").
- Construct LavaTrackerSdk(registry, ..., clonedProviderDao = fakeDao).
- Call sdk.multiSearch(SearchRequest("ubuntu"), providerIds = listOf("rutracker.clone.x")).

- Assert: `result.providerStatuses[0].state == SUCCESS` (was FAILURE/"not registered" before F.1).



Step 3: Test 2 — clone results tagged with clone synthetic id.

- Same setup; the fake source returns items with `trackerId = "rutracker"`.
- Assert: every `result.items[*].occurrences[*].providerId` is `"rutracker.clone.x"`, NOT `"rutracker"`.
- This is the user-visible state: the search-result list-cell badge reads `"RuTracker EU"`, not `"RuTracker.org"`.



Step 4: Falsifiability rehearsals documented in KDoc + executed pre-commit.

- For Test 1: revert `clientFor(id)` back to `registry.get(id, MapPluginConfig())`. Test fails with `IllegalArgumentException`.
- For Test 2: in `wrapSearchable`, return `src.search(request, page)` without the `.copy(items = items.map { ... })` re-tag. Test fails: items carry source id, not clone id.

Task 5 — CONTINUATION + commit + push

Files:

- Modify: `docs/CONTINUATION.md`

Steps:



\$0 update: Phase F.1 landed; F.2 (per-clone MirrorManager) owed.



Commit with both Bluff-Audit stamps.



Push to github + gitlab. Verify SHA convergence.